

FIT3155

SAMPLE EXAM PAPER

FIT3155 SAMPLE EXAM PAPER

Generated Sample Exam 04

INSTRUCTIONS

- This exam is worth 60 marks.

General exam technique

- Some students lose marks by not attempting all questions.
- Suppose you are likely to get 3/6 on a question after spending say 10 minutes on it. Spending another 10 minutes on the same question may gain you at most 3 additional marks. In contrast, spending those 10 minutes on a new question could earn you another 6 marks. The key point is: do not get stuck on one question.
- A good strategy is to first answer the questions you are most confident about before attempting the others. In other words, answering questions strictly serially during an exam can be suboptimal.
- Do not waste time answering an unrelated question in the hope of gaining partial marks. Marking will be based on the question that was actually asked.
- Where necessary, justify your answers with clear and concise explanations, without being overly wordy.

Question 1**(6=2+4 marks)**

For any string $S[1 \dots n]$, define the *Z-suffix value* $Zsuf_i(S)$, for each position $1 \leq i < n$, as the length of the longest substring of S ending at position i that is identical to a suffix of S .

Example: for $S[1 \dots 7] = \text{aabxaab}$, $Zsuf_3(S) = 3$, since $S[1 \dots 3] = \text{aab}$ is identical to the suffix $S[5 \dots 7] = \text{aab}$.

- (i) Compute $Zsuf_i(S)$ for every position $1 \leq i < n$ of the string

$S[1 \dots 6] = \text{babcab}$.

- (ii) Write pseudocode describing an efficient algorithm that takes any input string $S[1 \dots n]$ and outputs all of its Z-suffix values. State the worst-case time complexity of your algorithm.

(For this problem, you may assume that `zalg` is available as a built-in function that takes a string as input and returns the array of its Z-values. Do not write pseudocode for `zalg`.)

(Page intentionally left blank for working space)

Question 2**(6 marks)**

In the Boyer–Moore algorithm for exact matching of the pattern $pat[1 \dots m]$ against the text $txt[1 \dots n]$, suppose that during some iteration the pattern is aligned with $txt[j \dots j+m-1]$, and the right-to-left scan finds that *every* character matches, i.e., an occurrence of pat has been found starting at position j of the text.

To continue searching for further occurrences, the pattern is then shifted $m - mp[2]$ positions to the right, where $mp[2]$ denotes the length of the longest suffix of $pat[2 \dots m]$ that is identical to a prefix of pat .

Prove that this shift is safe, i.e., such a shift can never skip pat past any valid occurrence in txt .

(Page intentionally left blank for working space)

Question 3**(6=2+4 marks)**

Let $S[1 \dots n]$ be a string whose final character $S[n] = \$$ is a unique terminal symbol, lexicographically smaller than every other character. Let $SA[1 \dots n]$ be the suffix array of S , let $L[1 \dots n]$ be the Burrows–Wheeler Transform of S , and let $F[1 \dots n]$ denote the first column of the sorted cyclic rotation matrix (i.e., the characters of S in sorted order).

- (i) Write pseudocode that computes $L[1 \dots n]$ given only S and SA as input. State the worst-case time complexity of your procedure.
- (ii) Prove the following property: for every character c and every i , the i -th occurrence of c in L (reading top to bottom) and the i -th occurrence of c in F (reading top to bottom) correspond to the same occurrence of c in S .

(This is the rank-preservation property underpinning the LF mapping used in BWT inversion and backward search.)

(Page intentionally left blank for working space)

Question 4**(6=3+3 marks)**

Assume that a suffix tree for $S[1 \dots n]$ has *already* been constructed, with each edge label stored as a pair of indices $(start, end)$ into the string. You may assume the alphabet is of constant size.

- (i) Describe a preprocessing step that takes $O(n)$ time, after which the *number* of occurrences in S of any query pattern $pat[1 \dots m]$ can be reported in $O(m)$ time. Justify why the count your method reports is correct, and why the stated time bounds hold.
- (ii) Describe an $O(n)$ -time algorithm that finds the longest *repeated* substring of S , i.e., the longest substring that occurs at least twice in S . Justify the correctness of your algorithm.

(Page intentionally left blank for working space)

Question 5**(6=3+3 marks)**

- (i) A text of length 100 contains exactly five distinct symbols, with the following frequencies:

symbol	a	b	c	d	e
frequency	38	25	20	10	7

Construct a Huffman code for these symbols, showing the order in which subtrees are merged. Give a valid codeword for each symbol, and compute the total length (in bits) of the resulting Huffman encoding of the text.

- (ii) Work out the Elias-Omega binary codeword for the positive integer 100. Show each length component of your codeword.

(Page intentionally left blank for working space)

Question 6**(6=3+3 marks)**

- (i) Write pseudocode for an efficient algorithm that computes

$$a^b \bmod n$$

for positive integers a , b , n , using the method of repeated squaring. State, as a function of b , the number of modular multiplications your algorithm performs in the worst case.

- (ii) Using repeated squaring, compute

$$5^{21} \bmod 23.$$

Show the value of each successive squaring modulo 23, and indicate which of those values are multiplied together to obtain the final answer.

(Page intentionally left blank for working space)

Question 7**(6=3+3 marks)**

Let x be any node of degree k in a Fibonacci heap, and let $y_1, y_2, \dots, y_k$ denote the current children of x , in the order in which they were linked to x (earliest first). Recall that nodes become children of other nodes only through the consolidation step (which links two roots of *equal* degree), and that a non-root node is cut from its parent as soon as it loses a second child.

- (i) Prove that $\deg(y_i) \geq i - 2$ for all $2 \leq i \leq k$.
- (ii) Using part (i), prove that the subtree rooted at any node of degree k contains at least F_{k+2} nodes, where $F_0 = 0$, $F_1 = 1$, and $F_j = F_{j-1} + F_{j-2}$. Hence conclude that the maximum degree of any node in an n -node Fibonacci heap is $O(\log n)$.
(You may assume without proof that $\sum_{i=0}^k F_i = F_{k+2} - 1$ and that $F_{k+2} \geq \phi^k$, where $\phi = (1 + \sqrt{5})/2$.)

(Page intentionally left blank for working space)

Question 8**(6 marks)**

A stack data structure supports the following three operations:

- $\text{PUSH}(x)$: pushes x onto the stack, with actual cost 1;
- $\text{POP}()$: pops the top element, with actual cost 1;
- $\text{MULTIPOP}(k)$: pops $\min(k, \text{size})$ elements off the stack, with actual cost equal to the number of elements actually popped.

The stack is initially empty.

Using the *accounting* (banker's) method of amortized analysis, determine the amortized cost of each of the three operations. State the charge you assign to each operation, state the credit invariant that your scheme maintains, and verify that the total stored credit can never become negative over any sequence of operations starting from the empty stack.

(Page intentionally left blank for working space)

Question 9**(6=3+3 marks)**

Consider the following B-tree with minimum degree $t = 2$, so that every node stores between $t - 1 = 1$ and $2t - 1 = 3$ keys (the root is exempt from the lower bound):

- the root contains the keys $[20, 40]$;
- its three children are, from left to right, $[10, 15]$, $[25, 30]$, and $[50]$.

Use the following deletion conventions. A key in an internal node is deleted by replacing it with its in-order *predecessor* (the largest key in the child subtree immediately to its left), which is then deleted from its original node. If a deletion would leave a non-root node with fewer than $t - 1$ keys, then: (a) if an immediate sibling has more than $t - 1$ keys, *borrow* through the parent (the separating key in the parent moves down into the deficient node, and the sibling's key adjacent to the separator moves up into the parent); (b) otherwise, *merge* the deficient node with an immediate sibling and the separating key from the parent.

- (i) Delete key 50. Draw the resulting B-tree, clearly indicating the keys in every node.
- (ii) From the B-tree obtained in part (i), delete key 20. Draw the resulting B-tree, clearly indicating the keys in every node.

(Page intentionally left blank for working space)

Question 10**(6=3+3 marks)**

- (i) Convert the following linear program into the standard form used by the simplex method (a maximization objective, $\leq$ constraints only, and non-negative decision variables), and then rewrite each constraint as an equality by introducing slack variables. Do *not* solve the program.

$$\begin{array}{ll}\text{Minimize} & z = 2x_1 - 3x_2 \\ \text{Subject to} & x_1 + x_2 = 10, \\ & x_1 - 2x_2 \geq -4, \\ & x_1 \geq 0, \quad x_2 \geq 0.\end{array}$$

- (ii) Consider the linear program:

$$\begin{array}{ll}\text{Maximize} & z = 5x + 4y \\ \text{Subject to} & 2x + y \leq 8, \\ & x + 3y \leq 9, \\ & x \geq 0, \quad y \geq 0.\end{array}$$

Introduce slack variables and write down the *initial* simplex tableau. State the basic feasible solution that this tableau represents. Then identify the *entering* variable and, using the minimum-ratio test, the *leaving* variable for the first pivot of the simplex method. Do *not* perform the pivot.

(Page intentionally left blank for working space)

End of Exam